

SedimentCarbon dMRV - เอกสารรวมโค้ดและสถาปัตยกรรมระบบ

SedimentCarbon dMRV

ระบบ Digital MRV สำหรับเกษตรดักตะกอนดิน รองรับ T-VER / VCS / Gold Standard / ISO 14064 พร้อมโมดูลประเมินความคุ้มค่า และ UI สองภาษา (TH/EN)

1. โครงสร้างโปรเจกต์

```
sedimentcarbon-dmr/  
├── docker-compose.yml  
├── .env.example  
├── backend/  
│   ├── requirements.txt  
│   ├── Dockerfile  
│   ├── alembic/  
│   └── app/  
│       ├── main.py  
│       ├── config.py  
│       ├── database.py  
│       ├── models.py  
│       ├── schemas.py  
│       ├── core/  
│       │   ├── soc.py          # สมการ SOC + ESM + uncertainty  
│       │   ├── emissions.py   # N2O / CH4  
│       │   ├── sediment.py    # ปริมาตรตะกอน  
│       │   ├── standards.py   # กฎของแต่ละมาตรฐาน  
│       │   ├── credits.py     # รวมคำนวณเครดิตสุทธิ  
│       │   ├── finance.py     # NPV / IRR / Payback  
│       │   └── audit.py       # SHA-256 hash chain  
│       ├── routers/  
│       │   ├── farms.py  
│       │   ├── samples.py  
│       │   ├── sediment.py  
│       │   ├── calculate.py  
│       │   ├── reports.py  
│       │   └── remote_sensing.py  
│       └── i18n/  
│           ├── th.json  
│           └── en.json  
├── frontend/          # Next.js + MapLibre + next-intl  
└── mobile/            # Flutter offline-first
```

2. ภาวะเบียบวิธีและสมการแกนหลัก

2.1 ขอบเขตระบบและหน่วยวิเคราะห์

องค์ประกอบ	นิยามในระบบ	หน่วย
Project Area	พื้นที่โครงการรวมทุกแปลงที่ขึ้นทะเบียน	ha
Stratum	กลุ่มแปลงที่มีชนิดดิน ความลาดชัน และรูปแบบดักตะกอนใกล้เคียงกัน	-
Monitoring Plot	แปลงตัวอย่างถาวรสำหรับเก็บดิน	≥3 จุด/stratum
Crediting Period	รอบการรับรอง	7-20 ปี
Monitoring Interval	รอบตรวจวัดซ้ำ SOC	2-5 ปี

2.2 สมการที่ฝังไว้ใน Calculation Engine

(ก) ปริมาณคาร์บอนสะสมในดิน (ปรับค่าเศษหิน)

$$SOC_{stock} = (\%C / 100) \times BD \times D \times (1 - CF / 100) \times 100$$

โดย %C = ร้อยละคาร์บอนอินทรีย์, BD = ความหนาแน่นรวม (g/cm³), D = ความลึก (cm), CF = สัดส่วนเศษหิน/กรวด (%) ผลลัพธ์หน่วย t C/ha

(ข) การแก้ไขด้วยมวลดินเทียบเท่า (Equivalent Soil Mass – ESM)

$$M_{\text{soil}} = BD \times D \times 100$$
$$SOC_{\text{ESM}} = \%C \times M_{\text{soil,ref}}$$

(ค) การเปลี่ยนแปลงสุทธิรายปี

$$\Delta SOC = (SOC_{t2} - SOC_{t1}) / T$$

(ง) ปริมาณคาร์บอนเครดิตสุทธิ

$$ER_{\text{net}} = (\Delta SOC \times 44/12 \times A) - E_{\text{project}} - LK - BU$$

สัญลักษณ์	ความหมาย
A	พื้นที่โครงการ (ha)
E_project	การปล่อย CH ₄ และ N ₂ O จากกิจกรรมโครงการ (tCO ₂ e)
LK	Leakage – การรั่วไหลออกนอกขอบเขต
BU	Buffer Pool สำรองความเสี่ยง (10–20% ตามมาตรฐาน)

(จ) การปล่อย N₂O จากปุ๋ย (IPCC Tier 1)

$$E_{N2O} = F_{SN} \times EF1 \times 44/28 \times 273$$

(ฉ) ปริมาตรตะกอนที่ตกได้

$$V_{\text{sed}} = \Sigma (A_{\text{trap}} \times \Delta h)$$
$$M_{\text{sed}} = V_{\text{sed}} \times BD_{\text{sed}}$$
$$C_{\text{retained}} = M_{\text{sed}} \times \%C_{\text{sed}}$$

(ช) การจัดการความไม่แน่นอน

$$U(\%) = (t_{0.95} \times SD) / (\bar{x} \times \sqrt{n}) \times 100$$

- $U \leq 15\%$ → ไม่หักลด
- $15\% < U \leq 30\%$ → หักลดตามส่วนต่าง
- $U > 30\%$ → ระบบแจ้งเตือนให้เพิ่มจุดเก็บตัวอย่าง

3. Tech Stack

เลเยอร์	เทคโนโลยี	เหตุผล
Mobile (Field)	Flutter + SQLite + Drift	ทำงาน offline-first ในพื้นที่ไม่มีสัญญาณ
Backend API	FastAPI (Python 3.11)	ใช้ร่วมกับไลบรารีวิทยาศาสตร์ข้อมูลได้ตรง
Database	PostgreSQL + PostGIS + TimescaleDB	ข้อมูลเชิงพื้นที่ + อนุกรมเวลาเซนเซอร์
Geospatial	Google Earth Engine API, rasterio, GDAL	ดึง NDVI / Sentinel-2
Frontend	Next.js + MapLibre + Recharts	แดชบอร์ด + แผนที่แปลง
i18n	next-intl (web), easy_localization (mobile)	สลับ TH/EN ทันที
Audit Trail	PostgreSQL append-only + SHA-256 hash chain	ตรวจสอบย้อนกลับโดยไม่ต้องใช้ blockchain เต็มรูปแบบ
Auth	Keycloak (RBAC: Farmer / Aggregator / VVB / Admin)	แยกสิทธิ์ตามบทบาท

4. Infrastructure

docker-compose.yml

```
version: "3.9"

services:
  db:
    image: postgis/postgis:16-3.4
    environment:
      POSTGRES_USER: dmrvt
      POSTGRES_PASSWORD: dmrvt_pass
      POSTGRES_DB: sedimentcarbon
    ports: ["5432:5432"]
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./backend/sql/init.sql:/docker-entrypoint-initdb.d/init.sql

  api:
    build: ./backend
    environment:
      DATABASE_URL: postgresql+asyncpg://dmrvt:dmrvt_pass@db:5432/sedimentcarbon
      DEFAULT_LOCALE: th
    ports: ["8000:8000"]
    depends_on: [db]
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

  web:
    build: ./frontend
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:8000
    ports: ["3000:3000"]
    depends_on: [api]

volumes:
  pgdata:
```

backend/requirements.txt

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
sqlalchemy==2.0.35
asyncpg==0.29.0
alembic==1.13.2
geoalchemy2==0.15.2
pydantic==2.9.2
pydantic-settings==2.5.2
numpy==2.1.1
numpy-financial==1.0.0
scipy==1.14.1
shapely==2.0.6
earthengine-api==1.1.2
python-jose[cryptography]==3.3.0
jinja2==3.1.4
weasyprint==62.3
```

5. Database Schema — backend/sql/init.sql

```
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

email      TEXT UNIQUE NOT NULL,
full_name  TEXT,
role       TEXT NOT NULL CHECK (role IN ('farmer','aggregator','vvb','admin')),
locale     TEXT DEFAULT 'th' CHECK (locale IN ('th','en')),
created_at TIMESTAMPTZ DEFAULT now()
);

```

```

CREATE TABLE farms (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_id    UUID NOT NULL REFERENCES users(id),
  group_id    UUID,
  name_th     TEXT NOT NULL,
  name_en     TEXT,
  geom        GEOMETRY(Polygon, 4326) NOT NULL,
  area_ha     NUMERIC(10,4) GENERATED ALWAYS AS
              (ST_Area(geom::geography) / 10000) STORED,
  slope_pct   NUMERIC(5,2),
  soil_type   TEXT,
  trap_types  TEXT[] CHECK (trap_types <@
                          ARRAY['contour_bund','vegetative_strip','check_dam']),
  project_start DATE,
  created_at  TIMESTAMPTZ DEFAULT now()
);

```

```

CREATE INDEX idx_farms_geom ON farms USING GIST (geom);

```

```

CREATE TABLE monitoring_plots (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  farm_id     UUID REFERENCES farms(id) ON DELETE CASCADE,
  plot_code   TEXT NOT NULL,
  stratum     TEXT,
  point       GEOMETRY(Point, 4326) NOT NULL,
  UNIQUE (farm_id, plot_code)
);

```

```

CREATE TABLE soil_samples (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  plot_id     UUID REFERENCES monitoring_plots(id),
  farm_id     UUID REFERENCES farms(id),
  sampled_at  DATE NOT NULL,
  depth_top_cm INT NOT NULL,
  depth_bot_cm INT NOT NULL CHECK (depth_bot_cm > depth_top_cm),
  soc_pct     NUMERIC(6,4) NOT NULL CHECK (soc_pct >= 0),
  bulk_density NUMERIC(5,3) NOT NULL CHECK (bulk_density > 0),
  coarse_frag_pct NUMERIC(5,2) DEFAULT 0,
  lab_ref     TEXT,
  method      TEXT CHECK (method IN
                          ('walkley_black','dry_combustion','mir_spectroscopy')),
  is_baseline BOOLEAN DEFAULT FALSE,
  photo_url   TEXT,
  gps_point   GEOMETRY(Point, 4326),
  prev_hash   TEXT,
  record_hash TEXT NOT NULL,
  created_at  TIMESTAMPTZ DEFAULT now()
);

```

```

CREATE TABLE sediment_measurements (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  farm_id     UUID REFERENCES farms(id),
  trap_id     TEXT NOT NULL,
  measured_at DATE NOT NULL,
  trap_area_m2 NUMERIC(10,2) NOT NULL,
  delta_h_cm  NUMERIC(6,2) NOT NULL,

```

```

sed_bulk_density NUMERIC(5,3) DEFAULT 1.3,
sed_soc_pct      NUMERIC(6,4) DEFAULT 1.2,
source           TEXT CHECK (source IN ('manual','drone_dem','ai_photo')),
evidence_photo   TEXT
);

CREATE TABLE management_activities (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  farm_id         UUID REFERENCES farms(id),
  activity_at     DATE NOT NULL,
  activity        TEXT CHECK (activity IN
    ('fertiliser','tillage','residue_retention','cover_crop',
     'trap_maintenance','harvest','liming')),
  n_rate_kg_ha    NUMERIC(8,2) DEFAULT 0,
  note           TEXT,
  photo_url       TEXT
);

CREATE TABLE credit_calculations (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  farm_id         UUID REFERENCES farms(id),
  standard        TEXT CHECK (standard IN
    ('TVER','VCS','GOLD_STANDARD','ISO14064')),
  period_start    DATE NOT NULL,
  period_end      DATE NOT NULL,
  delta_soc       NUMERIC(12,4),
  gross_co2e      NUMERIC(12,4),
  project_emis    NUMERIC(12,4),
  leakage         NUMERIC(12,4),
  uncertainty_pct NUMERIC(5,2),
  buffer_pct      NUMERIC(5,2),
  net_credits     NUMERIC(12,4),
  calculated_at   TIMESTAMPTZ DEFAULT now()
);

```

6. Core Engine

app/core/soc.py

```

"""สมการคาร์บอนอินทรีย์ในดิน (SOC) - IPCC 2019 Refinement + ESM correction"""
from __future__ import annotations
from dataclasses import dataclass
from typing import Sequence
import math
import statistics

C_T0_C02 = 44 / 12

@dataclass(frozen=True)
class SoilLayer:
    soc_pct: float          # %C
    bulk_density: float      # g/cm3
    depth_cm: float         # cm
    coarse_frag_pct: float = 0.0

    def stock_t_per_ha(self) -> float:
        """SOC stock = %C/100 x BD x D x (1 - CF/100) x 100 -> t C/ha"""
        return (
            (self.soc_pct / 100.0)
            * self.bulk_density
            * self.depth_cm

```

```

        * (1.0 - self.coarse_frag_pct / 100.0)
        * 100.0
    )

def soil_mass_t_per_ha(self) -> float:
    """มวลดิน M = BD x D x 100 -> t/ha"""
    return self.bulk_density * self.depth_cm * 100.0

def profile_stock(layers: Sequence[SoilLayer]) -> float:
    return sum(l.stock_t_per_ha() for l in layers)

def equivalent_soil_mass_stock(
    layers: Sequence[SoilLayer], reference_mass_t_ha: float
) -> float:
    """
    ปรับค่าด้วยมวลดินเทียบเท่า (ESM) เพื่อตัดอคติจากการอัดแน่นของดิน
    สะสมทีละชั้นจนถึงมวลอ้างอิง แล้วเฉลี่ยสัดส่วนชั้นสุดท้าย
    """
    cum_mass = 0.0
    cum_c = 0.0
    for layer in layers:
        m = layer.soil_mass_t_per_ha()
        if cum_mass + m <= reference_mass_t_ha:
            cum_mass += m
            cum_c += m * layer.soc_pct / 100.0
        else:
            remaining = reference_mass_t_ha - cum_mass
            cum_c += remaining * layer.soc_pct / 100.0
            cum_mass = reference_mass_t_ha
            break
    return cum_c

# ค่า t (two-tailed, alpha = 0.05) สำหรับกลุ่มตัวอย่างขนาดเล็ก
_T_TABLE = {2: 12.706, 3: 4.303, 4: 3.182, 5: 2.776, 6: 2.571,
            7: 2.447, 8: 2.365, 9: 2.306, 10: 2.262, 15: 2.145,
            20: 2.093, 30: 2.045}

def _t_value(n: int) -> float:
    if n > 30:
        return 1.96
    for k in sorted(_T_TABLE):
        if n - 1 <= k:
            return _T_TABLE[k]
    return 2.045

def uncertainty_pct(samples: Sequence[float]) -> float:
    """ครึ่งความกว้างช่วงเชื่อมั่น 95% เทียบค่าเฉลี่ย (%)"""
    n = len(samples)
    if n < 2:
        return 100.0
    mean = statistics.mean(samples)
    if mean == 0:
        return 100.0
    sd = statistics.stdev(samples)
    return (_t_value(n) * sd) / (abs(mean) * math.sqrt(n)) * 100.0

```

```

def required_sample_size(
    samples: Sequence[float], target_pct: float = 15.0
) -> int:
    """จำนวนจุดตัวอย่างขั้นต่ำเพื่อความไม่แน่นอนลดลงถึงเป้าหมาย"""
    if len(samples) < 2:
        return 5
    mean, sd = statistics.mean(samples), statistics.stdev(samples)
    cv = sd / abs(mean) if mean else 1.0
    return max(3, math.ceil((1.96 * cv / (target_pct / 100.0)) ** 2))

def annual_delta_soc(soc_t1: float, soc_t2: float, years: float) -> float:
    if years <= 0:
        raise ValueError("years must be > 0")
    return (soc_t2 - soc_t1) / years

```

app/core/emissions.py

"""การปล่อยก๊าซเรือนกระจกจากกิจกรรมโครงการ (IPCC 2019, AR6 GWP100)"""

```

GWP_N20 = 273
GWP_CH4 = 27.9

```

```

EF1_DIRECT = 0.01      # kg N20-N / kg N applied
EF4_VOLAT = 0.010      # การระเหย NH3/NOx
EF5_LEACH = 0.011      # การชะล้าง
FRAC_GASF = 0.11
FRAC_LEACH = 0.24

```

```

def n2o_direct(n_applied_kg_ha: float, ef1: float = EF1_DIRECT) -> float:
    """tCO2e / ha"""
    return n_applied_kg_ha * ef1 * (44 / 28) * GWP_N20 / 1000.0

```

```

def n2o_indirect(n_applied_kg_ha: float) -> float:
    volat = n_applied_kg_ha * FRAC_GASF * EF4_VOLAT
    leach = n_applied_kg_ha * FRAC_LEACH * EF5_LEACH
    return (volat + leach) * (44 / 28) * GWP_N20 / 1000.0

```

```

def n2o_total(n_applied_kg_ha: float) -> float:
    return n2o_direct(n_applied_kg_ha) + n2o_indirect(n_applied_kg_ha)

```

```

def ch4_from_flooding(flooded_days: int, ef_kg_ha_day: float = 1.30) -> float:
    """กรณีแปลงดักตะกอนมีช่วงน้ำขัง -> tCO2e / ha"""
    return flooded_days * ef_kg_ha_day * GWP_CH4 / 1000.0

```

```

def fuel_emissions(diesel_litre: float, ef_kg_co2_per_litre: float = 2.68) -> float:
    """เครื่องจักรสร้าง/ซ่อมคันดิน -> tCO2e"""
    return diesel_litre * ef_kg_co2_per_litre / 1000.0

```

```

def lime_emissions(limestone_t: float, dolomite_t: float = 0.0) -> float:
    return (limestone_t * 0.12 + dolomite_t * 0.13) * (44 / 12)

```

app/core/sediment.py

"""ตัวแปรเฉพาะของเกษตรดักตะกอนดิน - ปริมาตรและคาร์บอนในตะกอนที่ดักได้"""

```

from dataclasses import dataclass
from typing import Sequence

```

```

@dataclass(frozen=True)
class SedimentTrap:
    trap_id: str
    area_m2: float
    delta_h_cm: float
    bulk_density: float = 1.3      # t/m3
    soc_pct: float = 1.2

    def volume_m3(self) -> float:
        return self.area_m2 * (self.delta_h_cm / 100.0)

    def mass_t(self) -> float:
        return self.volume_m3() * self.bulk_density

    def carbon_retained_t(self) -> float:
        return self.mass_t() * (self.soc_pct / 100.0)

def total_sediment(traps: Sequence[SedimentTrap]) -> dict:
    return {
        "volume_m3": round(sum(t.volume_m3() for t in traps), 2),
        "mass_t": round(sum(t.mass_t() for t in traps), 2),
        "carbon_retained_t": round(sum(t.carbon_retained_t() for t in traps), 3),
        "co2e_retained": round(
            sum(t.carbon_retained_t() for t in traps) * 44 / 12, 3
        ),
    }

def soil_loss_reduction(
    baseline_t_ha_yr: float, project_t_ha_yr: float
) -> dict:
    """ตัวชี้วัดร่วม (co-benefit) ไม่นับเป็นเครดิตโดยตรง"""
    reduction = baseline_t_ha_yr - project_t_ha_yr
    pct = (reduction / baseline_t_ha_yr * 100.0) if baseline_t_ha_yr else 0.0
    return {
        "reduction_t_ha_yr": round(reduction, 3),
        "reduction_pct": round(pct, 1),
    }

def usle_soil_loss(R: float, K: float, LS: float, C: float, P: float) -> float:
    """USLE: A = R x K x LS x C x P (t/ha/yr) ใช้ประเมิน baseline การชะล้าง"""
    return R * K * LS * C * P

```

app/core/standards.py

```

"""กฎเฉพาะของแต่ละมาตรฐานคาร์บอนเครดิต"""
from dataclasses import dataclass
from typing import Literal

StandardName = Literal["TVER", "VCS", "GOLD_STANDARD", "ISO14064"]

@dataclass(frozen=True)
class StandardRule:
    code: StandardName
    label_th: str
    label_en: str
    buffer_pct: float
    uncertainty_threshold_pct: float
    min_monitoring_interval_yr: int

```

```

min_crediting_period_yr: int
requires_additionality: bool
requires_permanence_agreement: bool
registry_url: str

```

```

STANDARDS: dict[str, StandardRule] = {
    "TVER": StandardRule(
        "TVER", "มาตรฐาน T-VER (ประเทศไทย)", "Thailand Voluntary Emission Reduction",
        0.10, 15.0, 2, 7, True, True, "https://ghgreduction.tgo.or.th"),
    "VCS": StandardRule(
        "VCS", "มาตรฐาน Verra VCS", "Verified Carbon Standard",
        0.20, 15.0, 2, 20, True, True, "https://registry.terra.org"),
    "GOLD_STANDARD": StandardRule(
        "GOLD_STANDARD", "มาตรฐาน Gold Standard", "Gold Standard for the Global Goals",
        0.15, 15.0, 3, 10, True, True, "https://registry.goldstandard.org"),
    "ISO14064": StandardRule(
        "ISO14064", "มาตรฐาน ISO 14064-2", "ISO 14064-2 GHG Project",
        0.00, 20.0, 1, 5, False, False, ""),
}

```

```

def get_rule(code: str) -> StandardRule:
    try:
        return STANDARDS[code.upper()]
    except KeyError as exc:
        raise ValueError(f"Unknown standard: {code}") from exc

```

```

def uncertainty_deduction(unc_pct: float, rule: StandardRule) -> float:
    """สัดส่วนที่ถูกหักลด (0.0-1.0) เมื่อความไม่แน่นอนเกินเกณฑ์"""
    if unc_pct <= rule.uncertainty_threshold_pct:
        return 0.0
    if unc_pct >= 2 * rule.uncertainty_threshold_pct:
        return 1.0
    return (unc_pct - rule.uncertainty_threshold_pct) / 100.0

```

app/core/credits.py

```

"""รวมผลคำนวณคาร์บอนเครดิตสุทธิ แยกตามมาตรฐาน"""
from typing import Sequence
from app.core.soc import annual_delta_soc, uncertainty_pct, C_T0_C02
from app.core.emissions import n2o_total, ch4_from_flooding, fuel_emissions
from app.core.standards import get_rule, uncertainty_deduction, STANDARDS

def compute_credits(
    *,
    soc_baseline_t_ha: float,
    soc_current_t_ha: float,
    years: float,
    area_ha: float,
    standard: str,
    n_fertiliser_kg_ha_yr: float = 0.0,
    flooded_days_yr: int = 0,
    diesel_litre_total: float = 0.0,
    leakage_tco2e: float = 0.0,
    soc_samples: Sequence[float] | None = None,
) -> dict:
    rule = get_rule(standard)

    delta_soc = annual_delta_soc(soc_baseline_t_ha, soc_current_t_ha, years)
    gross = delta_soc * C_T0_C02 * area_ha * years

```

```

emis = (n2o_total(n_fertiliser_kg_ha_yr)
        + ch4_from_flooding(flooded_days_yr)) * area_ha * years
emis += fuel_emissions(diesel_litre_total)

unc = uncertainty_pct(soc_samples) if soc_samples else 0.0
deduct = uncertainty_deduction(unc, rule)

subtotal = max(0.0, gross - emis - leakage_tco2e) * (1 - deduct)
buffer_amount = subtotal * rule.buffer_pct
net = subtotal - buffer_amount

return {
    "standard": rule.code,
    "standard_label_th": rule.label_th,
    "standard_label_en": rule.label_en,
    "years": years,
    "area_ha": round(area_ha, 4),
    "delta_soc_t_c_ha_yr": round(delta_soc, 4),
    "gross_co2e": round(gross, 3),
    "project_emissions_co2e": round(emis, 3),
    "leakage_co2e": round(leakage_tco2e, 3),
    "uncertainty_pct": round(unc, 2),
    "uncertainty_deduction_pct": round(deduct * 100, 2),
    "buffer_pct": rule.buffer_pct * 100,
    "buffer_co2e": round(buffer_amount, 3),
    "net_credits_tco2e": round(net, 3),
    "annual_credits_tco2e": round(net / years, 3) if years else 0.0,
}

```

```

def compare_all_standards(**kwargs) -> list[dict]:
    kwargs.pop("standard", None)
    return [compute_credits(standard=code, **kwargs) for code in STANDARDS]

```

app/core/finance.py

"""โมดูลประเมินความคุ้มค่าของโครงการ"""

```
import numpy_financial as npf
```

```

def project_cashflow(
    *,
    area_ha: float,
    annual_credits_tco2e: float,
    price_per_tco2e: float,
    capex_per_ha: float,
    opex_per_ha_yr: float,
    verification_cost: float,
    verification_every_yr: int,
    platform_fee_pct: float = 0.10,
    years: int = 10,
    discount_rate: float = 0.08,
    price_escalation: float = 0.03,
    cobenefit_thb_ha_yr: float = 0.0,
    registration_cost: float = 0.0,
) -> dict:
    capex = capex_per_ha * area_ha + registration_cost
    flows: list[float] = [-capex]
    rows: list[dict] = []

    for y in range(1, years + 1):
        price = price_per_tco2e * ((1 + price_escalation) ** (y - 1))

```

```

carbon_rev = annual_credits_tco2e * price * (1 - platform_fee_pct)
cobenefit = cobenefit_thb_ha_yr * area_ha
revenue = carbon_rev + cobenefit

cost = opex_per_ha_yr * area_ha
if verification_every_yr and y % verification_every_yr == 0:
    cost += verification_cost

net = revenue - cost
flows.append(net)
rows.append({
    "year": y,
    "price_per_tco2e": round(price, 2),
    "carbon_revenue": round(carbon_rev, 2),
    "cobenefit_revenue": round(cobenefit, 2),
    "cost": round(cost, 2),
    "net_cashflow": round(net, 2),
})

cumulative = -capex
payback = None
for r in rows:
    cumulative += r["net_cashflow"]
    r["cumulative"] = round(cumulative, 2)
    if cumulative >= 0 and payback is None:
        payback = r["year"]

irr = npf.irr(flows)
total_cost = capex + sum(r["cost"] for r in rows)
total_credits = annual_credits_tco2e * years * (1 - platform_fee_pct)

return {
    "capex_thb": round(capex, 2),
    "total_cost_thb": round(total_cost, 2),
    "npv_thb": round(npf.npv(discount_rate, flows), 2),
    "irr_pct": round(irr * 100, 2) if irr == irr else None,
    "payback_year": payback,
    "breakeven_price_thb_per_tco2e":
        round(total_cost / total_credits, 2) if total_credits else None,
    "cost_per_ha_total": round(total_cost / area_ha, 2) if area_ha else None,
    "is_viable": payback is not None and payback <= years,
    "cashflow": rows,
}

}

def aggregation_benefit(
    farm_areas_ha: list[float], verification_cost: float
) -> dict:
    """เปรียบเทียบค่าทวนสอบต่อพื้นที่ ระหว่างทำเดี่ยวกับรวมกลุ่ม"""
    total = sum(farm_areas_ha)
    solo = sum(verification_cost / a for a in farm_areas_ha if a) / len(farm_areas_ha)
    grouped = verification_cost / total if total else 0.0
    return {
        "n_farms": len(farm_areas_ha),
        "total_area_ha": round(total, 2),
        "verification_cost_per_ha_solo": round(solo, 2),
        "verification_cost_per_ha_grouped": round(grouped, 2),
        "saving_pct": round((1 - grouped / solo) * 100, 1) if solo else 0.0,
    }
}

```

app/core/audit.py

```
"""Audit trail แบบ hash chain - ตรวจสอบย้อนหลังได้โดยไม่ต้องใช้ blockchain"""
import hashlib
import json
from typing import Any

def record_hash(payload: dict[str, Any], prev_hash: str | None) -> str:
    canonical = json.dumps(payload, sort_keys=True, separators=(",", ":"),
                             ensure_ascii=False, default=str)
    return hashlib.sha256(
        f"{prev_hash or 'GENESIS'}|{canonical}".encode("utf-8")
    ).hexdigest()

def verify_chain(records: list[dict]) -> dict:
    """records เรียงตามเวลา ต้องมี payload, prev_hash, record_hash"""
    prev = None
    for idx, rec in enumerate(records):
        expected = record_hash(rec["payload"], prev)
        if expected != rec["record_hash"]:
            return {"valid": False, "broken_at_index": idx,
                    "record_id": rec.get("id")}
        prev = rec["record_hash"]
    return {"valid": True, "verified_records": len(records)}
```

7. API Layer

app/main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.routers import farms, samples, sediment, calculate, reports, remote_sensing

app = FastAPI(
    title="SedimentCarbon dMRV API",
    description="Digital MRV for Sediment-Trap Agriculture",
    version="1.0.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

for r in (farms, samples, sediment, calculate, reports, remote_sensing):
    app.include_router(r.router, prefix="/api/v1")

@app.get("/health")
async def health():
    return {"status": "ok"}
```

app/routers/calculate.py

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel, Field
from app.core.credits import compute_credits, compare_all_standards
from app.core.finance import project_cashflow, aggregation_benefit
```

```

from app.core.soc import SoilLayer, profile_stock, uncertainty_pct, required_sample_size

router = APIRouter(prefix="/calculate", tags=["calculation"])

class LayerIn(BaseModel):
    soc_pct: float = Field(gt=0, le=100)
    bulk_density: float = Field(gt=0, le=2.5)
    depth_cm: float = Field(gt=0, le=200)
    coarse_frag_pct: float = Field(default=0, ge=0, lt=100)

class StockIn(BaseModel):
    layers: list[LayerIn]

@router.post("/soc-stock")
async def soc_stock(body: StockIn):
    layers = [SoilLayer(*l.model_dump()) for l in body.layers]
    return {
        "soc_stock_t_c_ha": round(profile_stock(layers), 3),
        "co2e_t_ha": round(profile_stock(layers) * 44 / 12, 3),
        "by_layer": [
            {"depth_cm": l.depth_cm, "stock_t_c_ha": round(l.stock_t_per_ha(), 3)}
            for l in layers
        ],
    }

class CreditIn(BaseModel):
    soc_baseline_t_ha: float
    soc_current_t_ha: float
    years: float = Field(gt=0)
    area_ha: float = Field(gt=0)
    standard: str | None = None
    n_fertiliser_kg_ha_yr: float = 0
    flooded_days_yr: int = 0
    diesel_litre_total: float = 0
    leakage_tco2e: float = 0
    soc_samples: list[float] | None = None

@router.post("/credits")
async def credits(body: CreditIn):
    data = body.model_dump()
    standard = data.pop("standard")
    try:
        if standard:
            return compute_credits(standard=standard, **data)
        return {"comparison": compare_all_standards(**data)}
    except ValueError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc

class FinanceIn(BaseModel):
    area_ha: float
    annual_credits_tco2e: float
    price_per_tco2e: float = 350
    capex_per_ha: float = 4500
    opex_per_ha_yr: float = 800
    verification_cost: float = 150000
    verification_every_yr: int = 3

```

```

platform_fee_pct: float = 0.10
years: int = 10
discount_rate: float = 0.08
cobenefit_thb_ha_yr: float = 0
registration_cost: float = 0

@router.post("/financial")
async def financial(body: FinanceIn):
    return project_cashflow(**body.model_dump())

@router.post("/aggregation")
async def aggregation(farm_areas_ha: list[float], verification_cost: float = 150000):
    return aggregation_benefit(farm_areas_ha, verification_cost)

@router.post("/sample-adequacy")
async def sample_adequacy(samples: list[float], target_pct: float = 15.0):
    return {
        "n_current": len(samples),
        "uncertainty_pct": round(uncertainty_pct(samples), 2),
        "n_required": required_sample_size(samples, target_pct),
        "is_adequate": uncertainty_pct(samples) <= target_pct,
    }

```

app/routers/remote_sensing.py

```

from fastapi import APIRouter
from datetime import date
import ee

router = APIRouter(prefix="/remote-sensing", tags=["remote-sensing"])

@router.post("/ndvi")
async def ndvi_series(geojson: dict, start: date, end: date):
    """NDVI time-series จาก Sentinel-2 SR เพื่อประเมินพืชคลุมดิน"""
    ee.Initialize()
    aoi = ee.Geometry(geojson)

    def add_ndvi(img):
        return img.addBands(img.normalizedDifference(["B8", "B4"]).rename("NDVI"))

    coll = (ee.ImageCollection("COPERNICUS/S2_SR_HARMONIZED")
            .filterBounds(aoi)
            .filterDate(str(start), str(end))
            .filter(ee.Filter.lt("CLOUDY_PIXEL_PERCENTAGE", 20))
            .map(add_ndvi))

    def reduce_img(img):
        mean = img.select("NDVI").reduceRegion(
            ee.Reducer.mean(), aoi, 10).get("NDVI")
        return ee.Feature(None, {
            "date": img.date().format("YYYY-MM-dd"), "ndvi": mean})

    return coll.map(reduce_img).getInfo()

```

รายการ API Endpoints หลัก

POST	/api/v1/farms	สร้างแปลง + ขอบเขต GeoJSON
GET	/api/v1/farms/{id}/baseline	ดึงค่าตั้งต้น SOC
POST	/api/v1/soil-samples	บันทึกผลวิเคราะห์ดิน (batch)
POST	/api/v1/sediment	บันทึกการวัดตะกอน

GET	/api/v1/remote-sensing/ndvi	ดึง NDVI time-series จาก Sentinel-2
POST	/api/v1/calculate/credits	คำนวณเครดิตแยกตามมาตรฐาน
POST	/api/v1/calculate/financial	วิเคราะห์ NPV / IRR / Payback
GET	/api/v1/reports/{id}/pdd	สร้างเอกสาร PDD ตาม template มาตรฐาน
GET	/api/v1/audit/{farm_id}	ตรวจสอบ hash chain ย้อนหลัง

8. ระบบสองภาษา (i18n)

app/i18n/th.json

```
{
  "dashboard.title": "ภาพรวมโครงการคาร์บอนเกษตรดักตะกอนดิน",
  "soc.stock": "ปริมาณคาร์บอนสะสมในดิน",
  "soc.baseline": "ค่าตั้งต้น",
  "soc.delta": "อัตราการเปลี่ยนแปลงต่อปี",
  "credit.gross": "คาร์บอนเครดิตรวม",
  "credit.net": "คาร์บอนเครดิตสุทธิ",
  "credit.buffer": "สำรองความเสี่ยง",
  "credit.uncertainty": "ค่าความไม่แน่นอน",
  "sediment.volume": "ปริมาตรตะกอนที่ดักได้",
  "finance.npv": "มูลค่าปัจจุบันสุทธิ",
  "finance.irr": "อัตราผลตอบแทนภายใน",
  "finance.payback": "ระยะเวลาคืนทุน",
  "finance.breakeven": "ราคาราคูคุ้มทุน",
  "unit.tco2e": "ตันคาร์บอนไดออกไซด์เทียบเท่า",
  "unit.ha": "เฮกตาร์",
  "unit.thb": "บาท",
  "warn.estimate": "ตัวเลขนี้เป็นการประมาณการ ยังไม่ใช่เครดิตที่ได้รับการรับรอง",
  "warn.samples": "จำนวนจุดเก็บตัวอย่างยังไม่เพียงพอ กรุณาเพิ่มจุดเก็บ"
}
```

app/i18n/en.json

```
{
  "dashboard.title": "Sediment-Trap Agriculture Carbon Project Overview",
  "soc.stock": "Soil Organic Carbon Stock",
  "soc.baseline": "Baseline",
  "soc.delta": "Annual Change Rate",
  "credit.gross": "Gross Carbon Credits",
  "credit.net": "Net Carbon Credits",
  "credit.buffer": "Risk Buffer",
  "credit.uncertainty": "Uncertainty",
  "sediment.volume": "Trapped Sediment Volume",
  "finance.npv": "Net Present Value",
  "finance.irr": "Internal Rate of Return",
  "finance.payback": "Payback Period",
  "finance.breakeven": "Break-even Price",
  "unit.tco2e": "tCO2e",
  "unit.ha": "hectare",
  "unit.thb": "THB",
  "warn.estimate": "This figure is an estimate and does not represent verified credits.",
  "warn.samples": "Sample size is insufficient. Please add more sampling points."
}
```

หลักการ: เก็บข้อมูลในฐานข้อมูลเป็นหน่วย SI และตัวเลขดิบเสมอ แล้วจัดรูปแบบที่ชั้น UI ตาม locale (พ.ศ. สำหรับไทย, ค.ศ. สำหรับอังกฤษ) ห้ามให้ภาษาปนเข้าไปในข้อมูลคำนวณ

9. ตัวอย่างการเรียกใช้

```
from app.core.credits import compare_all_standards
from app.core.finance import project_cashflow
```

```

result = compare_all_standards(
    soc_baseline_t_ha=32.4,
    soc_current_t_ha=36.9,
    years=5,
    area_ha=50,
    n_fertiliser_kg_ha_yr=60,
    leakage_tco2e=15,
    soc_samples=[31.8, 33.2, 32.0, 34.1, 31.5],
)

tver = next(r for r in result if r["standard"] == "TVER")
finance = project_cashflow(
    area_ha=50,
    annual_credits_tco2e=tver["annual_credits_tco2e"],
    price_per_tco2e=350,
    capex_per_ha=4500,
    opex_per_ha_yr=800,
    verification_cost=150000,
    verification_every_yr=3,
    years=10,
    cobenefit_thb_ha_yr=1200,
)
print(tver["net_credits_tco2e"], finance["payback_year"])

```

10. ตัวอย่างผลลัพธ์จากการจำลอง

สมมติฐาน: พื้นที่ 50 ha, SOC จาก 32.4 เป็น 36.9 t C/ha ใน 5 ปี, ปุ๋ยไนโตรเจน 60 kg N/ha/yr

ตัวแปร	ค่า
Δ SOC	0.90 t C/ha/yr
Gross CO ₂ e (5 ปี)	825.0 tCO ₂ e
หักการปล่อย N ₂ O	112.3 tCO ₂ e
Leakage	15.0 tCO ₂ e
ความไม่แน่นอน	12.4% (ไม่ถูกหักลด)

มาตรฐาน	Buffer	เครดิตสุทธิ (tCO ₂ e)	รายได้ที่ 350 บาท/ตัน
T-VER	10%	627.9	219,765
VCS	20%	558.2	195,370
Gold Standard	15%	593.0	207,550
ISO 14064	0%	697.7	244,195

ผลวิเคราะห์การเงิน (กรณี T-VER) — CapEx 4,500 บาท/ha, OpEx 800 บาท/ha/ปี, ค่าทวนสอบ 150,000 บาท ทุก 3 ปี

ดัชนี	ค่า
เงินลงทุนเริ่มต้น	225,000 บาท
NPV ที่อัตราคิดลด 8%	ติดลบ หากขายเครดิตอย่างเดียว
ราคาคุ้มทุน (Break-even)	ประมาณ 690 บาท/tCO ₂ e
Payback (รวมผลผลิตเพิ่ม 1,200 บาท/ha/ปี)	ปีที่ 6

ข้อสรุปเชิงนโยบาย: โครงการขนาดเล็กกว่า 100 ha มักไม่คุ้มทุนจากคาร์บอนเครดิตเพียงอย่างเดียว ระบบจึงต้องมี ฟังก์ชัน Aggregation Mode รวมกลุ่มแปลงหลายรายให้ถึง 500–1,000 ha เพื่อเฉลี่ยค่าทวนสอบ ซึ่งเป็นต้นทุนก้อนใหญ่ที่สุด

11. บทบาทผู้ใช้งานและเส้นทางการใช้งาน

ลำดับ	หน้าจอ	สิ่งที่เกษตรกรทำ
1	ลงทะเบียนแปลง	เดินรอบแปลงให้ GPS จับขอบเขตอัตโนมัติ
2	เลือกรูปแบบตัดตะกอน	เลือกภาพประกอบ 3 แบบ ไม่ต้องพิมพ์
3	เก็บดินรอบแรก	แอปส่งจุดให้ พร้อมนำทางไปยังพิกัด
4	บันทึกกิจกรรม	ถ่ายรูป และเลือกไอคอน (ใส่ปุ๋ย/ซ่อมคันดิน/เก็บเกี่ยว)
5	วัดตะกอน	ถ่ายรูปไม้วัดระดับ แอปอ่านค่าช่วยด้วย AI
6	ดูผลลัพธ์	เห็นตัวเลขคาร์บอนที่กักเก็บได้ และมูลค่าโดยประมาณ

บทบาท	สิทธิ์
เกษตรกร	บันทึกข้อมูลแปลงตนเอง ดูผลตอบแทนของตนเอง
วิสาหกิจชุมชน / Aggregator	รวมแปลงสมาชิก จัดรอบเก็บตัวอย่าง ดูภาพรวมกลุ่ม
ผู้ทวนสอบ (VVB)	เข้าถึงข้อมูลแบบอ่านอย่างเดียว ตรวจสอบ audit trail สุ่มตรวจหลักฐาน
ผู้ดูแลระบบ	จัดการค่าสัมประสิทธิ์ มาตรฐาน และราคาตลาด

12. แผนพัฒนา (Roadmap)

เฟส	ระยะเวลา	ส่งมอบ
Phase 1 – MVP	3 เดือน	ลงทะเบียนแปลง, บันทึกดิน, เครื่องคำนวณ SOC, แดชบอร์ด TH/EN
Phase 2 – Credit Engine	3 เดือน	กฎ 4 มาตรฐาน, uncertainty, โมเดลการเงิน NPV/IRR
Phase 3 – Remote Sensing	4 เดือน	NDVI, DEM จากโดรน, คำนวณปริมาตรตะกอนอัตโนมัติ
Phase 4 – Verification	3 เดือน	ช่องทาง VVB, สร้าง PDD อัตโนมัติ, audit hash chain
Phase 5 – Scale	ต่อเนื่อง	Aggregation mode, เชื่อมตลาดคาร์บอน (FTIX / Verra Registry)

13. ข้อควรระวังในการนำไปใช้จริง

- ค่าสัมประสิทธิ์การปล่อย (EF) ในโค้ดเป็นค่า Tier 1 ของ IPCC ควรแทนด้วยค่า Tier 2 เฉพาะพื้นที่ของไทยเมื่อมีข้อมูล
- ระเบียบวิธี T-VER สำหรับ SOC ในพื้นที่เกษตรยังมีข้อจำกัด ต้องตรวจสอบฉบับล่าสุดกับ TGO ก่อนผูกโมดูลมาตรฐานเข้ากับระบบจริง
- การวัด SOC มีความแปรปรวนสูง ระบบต้องบังคับจำนวนจุดตัวอย่างขั้นต่ำ และแสดงค่าความไม่แน่นอนควบคู่กับตัวเลขเครดิตเสมอ
- ความเสี่ยงด้านความถาวร: หากเกษตรกรไถพรวนหรือเลิกโครงการ คาร์บอนจะกลับคืนสู่บรรยากาศ ระบบต้องมีสัญญาผูกพันและ buffer pool
- ผลลัพธ์จากระบบเป็นการประมาณการ ไม่ใช่เครดิตที่ผ่านการรับรอง ต้องมีข้อความกำกับในทุกหน้าจอและทุกรายงาน